The Code-Freeze Rush in Kubernetes

Anonymous Author(s)

***Abstract*—Kubernetes, which has become critical cloud infrastructure, ships three releases a year on a fixed calendar. Before each release, a code freeze stops new work, and the weeks until the release are used to test and stabilize the code. The deadline triggers a rush, as contributors try to merge their code in time for the coming release. Code merged late also reaches users sooner after merging, so it has less time to be tested before release. Late code can therefore show more problems after release even if it is no worse than code merged earlier. We separate these two explanations with a causal design on 25,579 merged pull requests across 27 Kubernetes releases. We find that (i) on each of the last 2 days before the freeze, 3 to 4 times as many pull requests become ready to merge as on an ordinary day, and the rush moves when the deadline moves; (ii) pull requests ready in the final week are more than twice as likely as earlier ones to have a problem found after release; (iii) four common explanations, namely the kind of changes that land late, pressure on review, a crowded merge queue, and the freeze itself, account for little of this gap; and (iv) followed for the same 180 days from merge, late code has no more problems in total, but more of its problems are found after release, as its shorter time between merge and release, which we call soak time, predicts. Release teams could therefore target how long late code runs before release, and studies that count defects after release can measure every change over the same window from its merge.**

***Index Terms*—Release engineering, code freeze, causal inference, software defects, Kubernetes, mining software repositories**

I. Introduction

In recent years, large open-source projects have moved to time-based releases, shipping on a fixed calendar rather than whenever features are ready [1]. Kubernetes, the container orchestration system, has shipped three minor releases a year since 2021 [2]. Each release has a code freeze period where every enhancement for the release, a new feature or change tracked by the release team, must be reviewed and approved by a fixed date, after which only approved changes for the release are merged until the freeze lifts, and the release ships about four weeks later [3], [4]. This fixed deadline invites a rush. The natural reading is that rushed code is worse, and this is not new to software engineering. Changes committed on Fridays [5] or late at night [6] were found to induce fixes more often, and time pressure has been linked to lower quality and skipped quality assurance [7].

These studies ask whether the timing of a change affects how often it fails. A separate question is whether timing affects when failures appear. At the release level, faster release cycles were found to change when users encounter bugs, but not how many they encounter [8]. For a single change, the time between merge and release may matter in the same way. Kubernetes treats this time as a period of testing and stabilization [4], and the weeks before release focus on "fixing bugs, eliminating test flakes and general release stabilization" [3]. Code merged late spends less of this time on the master branch before it reaches users. We therefore ask whether merging late moves a failure from before the release to after it. In causal terms, merging late (the treatment) shortens the time until release, a mediator, that is, a step between the treatment and the outcome. A shorter time could make a problem after release (the outcome) more likely even if the code is no worse (Section III-C).

Using 25,579 pull requests across 27 Kubernetes releases (v1.11–v1.37), we first describe how the code freeze shapes when code is merged, and how merging close to the freeze relates to problems after release. A *problem* is a later revert or fix of a pull request. We then test four common explanations for why late code has more problems after release, and finally follow every pull request for the same period from merge, to see whether late code fails more often or is simply found failing later.

We answer 2 research questions:

RQ1 *How does the code-freeze rush unfold?*

RQ2 *Does late code fail more often, or are its failures found later?*

We find that the rush is concentrated in the final 48 hours before the freeze, moves when the deadline moves, and has grown since 2021. The share of pull requests with a problem after release rises steadily as the freeze approaches, and *late* pull requests, those ready to merge in the final week, are more than twice as likely as earlier ones to have one (**RQ1**).

Late pull requests are larger, and more heavily reviewed, yet none of the four common explanations accounts for more than about 5% of their higher rate of problems after release (**RQ2**). When we follow every pull request for the same 180 days after merge and account for the kind of changes that land late, we see no sign that late code has more problems in total. More of its problems, however, are found after release, which matches its shorter soak time, the time between merge and release (**RQ2**).

II. Background & Related Work

The Kubernetes release cycle. A pull request to Kubernetes merges only after a reviewer applies the `lgtm` ("looks good to me") label and an approver applies the `approved` label; a merge bot then merges labeled pull requests once their tests pass. Before code freeze, every enhancement for the release must carry both labels, and because merges wait in a queue, approved pull requests may merge shortly after the deadline [3]. After the freeze, the release team accepts late enhancements only as exceptions, which should be "well-tested" and low risk [3]. The project describes its release process as ending in a *stabilization* phase, and the roughly

two weeks between the end of code freeze and the release are for resolving "all remaining critical issues" [4]. Release engineering research has long argued that such processes shape software quality [9]. Development is organized in special interest groups (SIGs) that own areas of the code base; contributors interact far more often within an area than across areas, and issues that span several areas take longer to resolve [10].

Timing and defects. Śliwerski et al. found that changes committed on Fridays were more likely to induce fixes [5], and Eyolfson et al. found that commits made late at night were buggier [6]. Time pressure more broadly has been linked to lower quality and skipped quality assurance [7]. At the level of releases, Khomh et al. found that after Firefox moved to rapid releases, users met bugs earlier and bugs were fixed faster [8], and Mäntylä et al. showed that shorter release cycles change how much and what kind of testing happens before release [11].

Measuring problems. Defect studies often link a fix back to the change that introduced it [12], an approach whose links are known to be noisy [13], [14], and many count the defects found in a window after release [15]. The time between a change and the fix of a defect it introduced varies widely [16]. Reverts are a second signal: a revert removes a change, often but not always because it broke something [17]. Review coverage and participation have been related to such post-release defects [18].

Causal methods in software engineering. Causal inference is increasingly used in software engineering [19]. Furia et al. show that a causal analysis of the same data can reach different conclusions than an associational one [20], and a recent roadmap calls for counterfactual analyses with explicit assumptions and placebo checks [21]. Section III-C states our causal question, the quantities we estimate, and the assumptions under which they are causal.

III. Methodology

A. Dataset Selection

We collect every pull request merged into `kubernetes/kubernetes` across 27 releases (v1.11 to v1.37), whose code freezes fall between June 2018 and July 2026.

For each release, we take the date and time of its code freeze and the date of its release from the release team's published schedule [22]. The merge queue can delay a merge past the deadline, so we date each pull request by when it becomes **ready**, not when it merges. A pull request becomes ready when it has both the `lgtm` and `approved` labels. We rebuild each label history from the pull request's timeline and check it against the pull request's current labels. We keep pull requests that become ready within 12 weeks before a code freeze, assign each to that release, and exclude those opened by bots. Pull requests ready in the final 7 days are **late**, and those ready 2 to 12 weeks before the freeze are **early**.

This gives 25,579 pull requests, 4,035 of them late.

B. Identifying Problems

A pull request has a **problem** when a later pull request reverts or fixes it. We use the term "problem" because a revert shows that a change was pulled out, not always that it was faulty.

Reverts: We take as reverts the pull requests whose title starts with "Revert" or whose branch was created by GitHub's revert button. We then link each revert to the pull request it undoes, using any one of three clues: the original's pull request number in the revert's title, branch name or description; the original's title quoted in the revert's title; or the commit recorded by `git revert`. We keep only reverts that undo the original: those that change at least one of the original's files with the additions and deletions swapped.

Fixes: A fix is a later pull request that repairs a defect introduced by an earlier one, the original. To find fixes, we first look for later pull requests that reference the original by its number. Pull requests reference each other for many reasons, so we keep only those that use words such as "regression", "broke", or "introduced in" and that change at least one file the original changed.

One author then reviews every remaining pair inside the windows in which we measure problems (Sections III-D and III-E) and decides whether the later pull request is a fix. The author sees the pairs in random order, with both pull requests' titles, descriptions, labels, and changed files, but not their dates or whether the original was late. Only pairs the author confirms count as fixes; undecided pairs are left out.

We report every outcome with two sets of links. *All links* are the reverts and fixes found by the automatic steps above, as specified before the analysis. *Checked links* keep only verified reverts and the fixes the author confirmed. **[[Elijah: please check this definition of all links vs checked links (which revert checks belong to each set?), and please update the legend of Fig. 3(a) ("all links (planned)", "hand-checked") to these two names.]]**

C. Causal Question and Assumptions

We want to know whether becoming ready late causes more problems after release, and through which path. Figure 1 shows the causal structure we assume. The *treatment* is becoming ready late, in the final week before the freeze. The comparison group is early pull requests, ready 2 to 12 weeks before it. The *outcome* is a problem found after release.

Late readiness can lead to this outcome in 2 ways. On the *code path*, late code is worse: it contains more problems, so more problems are found after release. On the *soak path*, late code is no worse, but it reaches users sooner after it merges, so a problem it contains has less time to be found before release. The 2 paths predict different results. If the code path holds, late pull requests have more problems in total. If only the soak path holds, they have as many problems as early ones, but more of them are found after release.

Late changes may also differ from early ones, for example in size, and larger changes are more likely to introduce defects [23]. Such differences in composition, such as size,

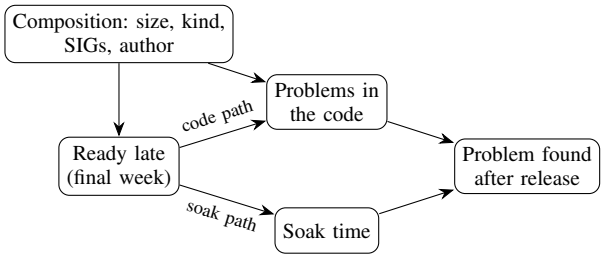


Fig. 1. Assumed causal structure. Late readiness can raise problems found after release through the code (code path) or through a shorter soak time, the time between merge and release (soak path). Composition confounds the code path. Pressure on review, a crowded merge queue, and the freeze response are candidate mechanisms of the code path.

kind, and author, affect both when a change becomes ready and how many problems it has. Composition is therefore a *confounder* of the code path, a factor that affects both the treatment and the outcome, and we adjust for it.

We estimate 3 quantities. We define the effect of the treatment as the difference between what would happen to the same pull request if it were late and if it were early [24]. For RQ1, we estimate the risk ratio, how many times as likely a late pull request is as an early one to have a problem found after release; we call the difference between late and early pull requests the *gap*. The risk ratio includes both paths. For RQ2, we estimate (i) the difference in problems in total within 180 days of merge after adjusting for composition, which measures the code path, and (ii) the share of late pull requests' problems found after release that soak time alone predicts, which measures the soak path. We estimate (i) with inverse-probability weights, which reweight early pull requests so that their composition matches that of late ones, combined with a doubly robust estimator, which also models the outcome and stays consistent if either the weights or the outcome model is correct [25].

These estimates are causal under 4 assumptions. First, given the composition variables, whether a pull request becomes ready late does not depend on the problems it would have; comparing pull requests by the same author also holds fixed what an author brings to every pull request. Second, for every late pull request, comparable early ones exist (overlap). Third, the problems of one pull request do not depend on when others merge; a crowded merge queue would break this assumption [26], so we test it as one of the explanations. Fourth, for the soak path, problems in late and early code take equally long to be found after merge, which we check by comparing their discovery times, the days from merge to the first problem (Section IV-B). As a falsification test [21], a placebo, a fake deadline 6 weeks before the freeze, should show no gap. **[[Elijah: please confirm how overlap is checked (e.g., weight distribution or trimming) and add one clause here.]]**

D. *How does the code-freeze rush unfold? (RQ1)*

The rush: For each release, we count the pull requests that became ready on each day. We then compare the last days before the freeze begins with an ordinary day 2 to 12 weeks earlier. To measure the size of the rush in each release, we divide the number of pull requests in the final week by the number in an ordinary earlier week. We also ask whether the rush grew when Kubernetes moved to three releases a year (v1.22). We compare the share of late pull requests before and after this move with a regression weighted by release size, which also accounts for how far into the cycle the freeze fell. Finally, we examine release v1.31, where the freeze date was moved two weeks later [27]. We compare the week before the original deadline with the week before the new one.

The impact: We place each pull request in a group by how long before the freeze it became ready: 9–12 weeks, 5–8 weeks, 3–4 weeks, week 2, days 3–7, or the final 48 hours. Our main outcome is whether a pull request had a problem found in the 90 days after its release. Because every pull request in a release ships on the same day, this window covers the same dates for all of them. We also count two outcomes measured from merge: a revert within 30 days, and a fix within 30 days. To compare late and early pull requests, we use the risk ratio (Section III-C). Pull requests in the same release share its release date and conditions, so our confidence intervals allow for this (clustered by release). Finally, we repeat the comparison with the placebo deadline six weeks before the real freeze: if the gap comes from the deadline, it should not appear there.

What changes as the freeze nears: Using the same six groups as above, from 9–12 weeks before the freeze to its final 48 hours, we describe what the pull requests in each group look like: their size (lines changed), their kind (the share labeled as features), how much review they received (review threads), how experienced their authors are (the author's prior pull requests), and how long they wait on the master branch before release (days from merge to release).

E. *Does late code fail more often, or are its failures found later? (RQ2)*

Four common explanations could account for why late pull requests have more problems after release than early ones: the kinds of changes that land late [23], [28], pressure on review [18], [29], a crowded merge queue [30], and the freeze itself [3].

What lands late (composition): Larger changes are more likely to introduce defects [23], [28]. If late changes differ from early ones in size or kind, the gap in problems may come from what lands late, not when it lands. To test this, we reweight early pull requests from weeks 5 to 12 before the freeze to match late ones on size label (the size category Kubernetes assigns to each pull request), lines and files changed, kind labels, number of SIGs, milestone, priority, the author's prior pull requests, pull request age, and release. We use inverse-probability weights and a doubly robust estimator (Section III-C), with a bootstrap over releases, which resamples whole releases 400 times to estimate uncertainty. **[[Elijah: early pull requests are weeks 2–12 elsewhere but weeks 5–12 here; please add the reason in one clause.]]** The share

of the gap removed by reweighting is the share explained by composition.

Pressure on review: Developers under time pressure tend to cut back on quality assurance [7], and how a change is reviewed is linked to whether it introduces defects [18], [29]. Reviewers facing the deadline may therefore approve late pull requests with less scrutiny. To test this, we compare late and early pull requests by the same author (author fixed effects, which compare each author only with themselves). We then estimate how much of the gap is explained by four review measures: review threads, reviews, whether `lgtm` was removed, and time from the last commit to readiness.

A crowded merge queue (congestion): When many changes merge at once, changes developed in parallel can conflict and break the build [31], even when each passes its tests alone [30]. A deadline packs merges into its final days, which may raise this risk. To test this, we look within the final week and compare pull requests in the same 6-hour bin before the freeze. We relate problems to the number of pull requests merged in the same area within 12 hours, total merges within 6 hours, and queue wait. Congestion is supported only if the same relation also appears in weeks 2 to 12, when no deadline is near.

The freeze itself (freeze response): During the freeze, only changes for release-blocking issues may merge [3]. A change that causes trouble may therefore be reverted to protect the release, even if it would otherwise get a later fix. Reverts also happen for reasons other than defects [17], [32], so more reverts need not mean more defects. Among pull requests with a problem within 30 days of merge, we test whether problems that surfaced during the freeze were more often reverted than fixed by a later pull request, after accounting for composition.

Soak time: The four explanations above focus on the code and how it was handled. None considers a simpler possibility: late code may just have less time for its problems to be found before release. Every pull request in a release ships on the same day, so one merged closer to the freeze has less **soak time**, the time between merge and release. Kubernetes uses this time for testing and stabilization [3], [4], reliability engineering plans how much testing time each part of a system needs before release [33], and release cycles are known to affect both how much testing happens before release [11] and how long finished work waits to reach users [34]. Because our main outcome counts problems only from the release onward, a late pull request could look worse even if it has no more problems than an early one.

To test this, we follow every pull request for the same 180 days from merge, regardless of when its release ships, and record whether its first problem was found before release or found after release. If late code is truly worse, late pull requests should have more problems in total. If its problems are only found later, the total should be the same, and more of them should fall after release. We apply the composition and author comparisons above to the total and to each part. We then compare how quickly problems are found after merge in late and early code, using the share found within each number of days. Finally, we estimate the share of late pull requests' problems that soak time alone would place after release. We take early pull requests' discovery times (the days from merge to first problem) and count how many would fall after each late pull request's release. We repeat this for each week before the freeze, each time using the discovery times of the other weeks.

IV. Results

A. *How does the code-freeze rush unfold? (RQ1)*

The rush: On an ordinary day 2 to 12 weeks before the freeze, about 10 pull requests per release become ready to merge. In the final week this rises, and on the last two days it reaches 30 and 39 (Fig. 2(a)): the final 48 hours alone carry about 70 pull requests per release. The rush follows the deadline. When the v1.31 freeze was moved two weeks later, twelve days before the original deadline, the week before that deadline brought only 48 pull requests, while the week before the new one brought 110, the most of any week in that release (Fig. 2(b)). The rush has also grown. Before 2021, the final week brought 1.8 times as many pull requests as an average earlier week; since the move to three releases a year, it brings 2.5 times as many (Fig. 2(c)), and the share of late pull requests rose by 5.2 percentage points [2.5, 8.0].

The impact: The closer to the freeze a pull request became ready, the more likely it was to have a problem found after release (Fig. 3(a)). The share rises steadily from 0.35% for pull requests ready 9 to 12 weeks before the freeze, through 0.84% for weeks 3 to 4, to 1.26% for days 3 to 7 and 1.84% for the final 48 hours. Checked links show the same rise at a lower level, from 0.06% to 0.50%. Overall, 1.5% of late pull requests and 0.6% of early ones had a problem found after release, a risk ratio of 2.55 [1.93, 3.38]; with checked links, the ratio is 2.93 [1.74, 4.95]. A placebo deadline six weeks before the freeze shows no such gap (0.71 [0.38, 1.33]). The gap did not clearly grow with the rush after the move to three releases a year (+0.2 percentage points [−0.6, 0.9]; +0.6 [0.2, 1.0] with checked links), and v1.31 had too few problems to compare its two deadline weeks (one each).

Reverts behave differently (Fig. 3(b)). The share reverted within 30 days of merge stays between 0.4% and 0.6% from 12 weeks before the freeze until its final 48 hours, then doubles to 1.2%. Overall, late pull requests were more often reverted within 30 days (1.57 [1.09, 2.26]), but not more often fixed within 30 days (1.28 [0.98, 1.67]; with checked links, 1.03 [0.54, 1.95]).

What changes as the freeze nears: Late pull requests are different (Fig. 3(c)–(f)). From 9–12 weeks before the freeze to the final 48 hours, the median pull request grows from 28 to 94 lines changed, the share of features rises from 11% to 36%, and review threads per pull request rise from 2.4 to 7.8. The authors are equally experienced, with a median of 30 to 41 prior pull requests in every group. The time code spends on the master branch before release falls from a median of 96 days to 28 days (Fig. 3(f)). Late code is thus both different and

TABLE I
CANDIDATE EXPLANATIONS FOR THE GAP IN PROBLEMS FOUND AFTER RELEASE, WITH ALL LINKS AND WITH CHECKED LINKS (SECTION III-B). PP: PERCENTAGE POINTS; IQR: INTERQUARTILE RANGE; 95% CONFIDENCE INTERVALS IN BRACKETS.

Explanation (test)	All links	Checked links	Explains?
The gap: late vs early (risk ratio)	2.55 [1.93, 3.38]	2.93 [1.74, 4.95]	–
What lands late: share of the gap removed by reweighting	1.2% [−6.7, 9.4]	0.5% [−29.1, 19.1]	No
Pressure on review: gap within the same author (pp)	1.09 [0.63, 1.54]	0.34 [0.10, 0.59]	
through review measures (pp)	0.06 [0.02, 0.10]	0.01 [−0.02, 0.03]	No
Crowded queue: same-area merges per IQR, final week (pp)	0.66 [0.06, 1.25]	0.40 [−0.09, 0.89]	
same, weeks 2–12 (pp)	0.03 [−0.15, 0.22]	0.03 [−0.09, 0.16]	No
Freeze response: reverted rather than fixed, in vs out of freeze (pp)	3.0 [−6.6, 12.6]	5.9 [−7.6, 19.4]	No

less tested before release; RQ2 asks which of the two explains the gap.

B. Does late code fail more often, or are its failures found later? (RQ2)

Candidate explanations: None of the four common explanations accounts for the gap, with either set of links (Table I). The kind of changes that land late explains about 1% of it: although late pull requests are larger and more often features, reweighting early pull requests to look like late ones leaves the gap almost unchanged. Pressure on review does not explain it either. The gap remains for the same author (+1.09 percentage points), and only about one in twenty of it runs through review; in fact, late pull requests receive more review, not less. A crowded queue shows a weak relation in the final week for merges in the same area, but no relation in weeks 2 to 12, so it does not meet our test for congestion. Finally, problems that surface during the freeze are not more often reverted (22.7% vs 20.8%; adjusted +3.0 percentage points [−6.6, 12.6]). Reweighting does, however, absorb much of the 30-day revert gap, which halves and can no longer be told apart from zero (+0.15 percentage points [−0.31, 0.48]).

Total vs timing: With checked links and followed for 180 days from merge, 2.1% of late and 1.5% of early pull requests had a problem. Once the kind of changes that land late is taken into account, the difference is −0.2 percentage points [−1.6, 0.7]: we find no sign that late code has more problems in total. The timing of those problems, however, differs. Late pull requests had more problems found after release (0.47% vs 0.18%; adjusted +0.33 percentage points [0.17, 0.50]), and, after adjustment, slightly fewer found before release (−0.5 [−2.0, 0.6]). The pattern holds for the same author (after release +0.32 [0.09, 0.54]) and with all links (total +0.76 [−0.49, 1.83]; after release +1.15 [0.78, 1.50]).

Discovery speed: Problems in late code surface as fast as problems in early code (Fig. 4(a)). In both groups, 72% of problems are found within 21 days of merge, and 87 to 88% within 60 days; in the first week, late code's problems are, if anything, found slightly sooner (58% vs 50%). What differs is when the release comes. Late code ships a median of 31 days after merge, when 77% of its problems have been found; early code ships after 66 days, when 89% have.

Timing prediction: Soak time predicts the shift. Given early pull requests' discovery times, 24% of late pull requests' problems should be found after release, and we observe 23%; for early pull requests, the prediction is 11% and we observe 12%. Across the weeks before the freeze, the observed share falls as soak time grows, from 23% in the final week to 3% in weeks 9 to 12, and stays within four percentage points of the prediction in every group (Fig. 4(b)). The steady rise in problems after release in Fig. 3(a), which begins two to three months before the freeze, long before any rush, is the pattern soak time predicts: a gradual rise as the time to release shrinks, rather than a jump at the deadline. Late code thus appears less tested before release, not worse.

V. DISCUSSION

The shape of the rush: The code-freeze rush in Kubernetes is concentrated in the final two days and growing: since the move to three releases a year, the final week brings two and a half times as many pull requests as an ordinary week. In v1.31, when the freeze moved two weeks later, the rush followed the new date, which indicates that the deadline sets when the rush happens. Such series are cheap to compute from a project's history, and they let release teams see how their calendar shapes when work arrives.

Less tested, not worse: Our findings suggest that the extra problems users meet after a release come from less testing time, not from worse code. This matters for what release teams do about the rush. Discouraging late merges may not help: in v1.31 the rush followed the deadline, so any deadline is likely to have a final-week rush. What release teams can set is how long the final week's code runs before release. A longer gap between freeze and release, or extra soak time for final-week merges, would act on the soak path. Companies that deploy continuously use a similar step: at Facebook, code that passes the automated tests first runs for the company's employees before it reaches all users [35]. The Kubernetes exception process already asks that late changes be "well-tested" [3]; soak time offers a measurable version of that request. One signal does peak at the deadline: pull requests ready in the final 48 hours are reverted within 30 days twice as often. These are the largest pull requests and most often features, and once the kind of changes that land late is taken into account, the final week's revert gap can no longer be told apart from zero.

Time pressure and review: Time pressure has been linked to lower quality and skipped quality assurance [7]. In Kubernetes, late pull requests received more review, not less, with three times as many review threads in the final 48 hours as two to three months before the freeze. Code review also finds

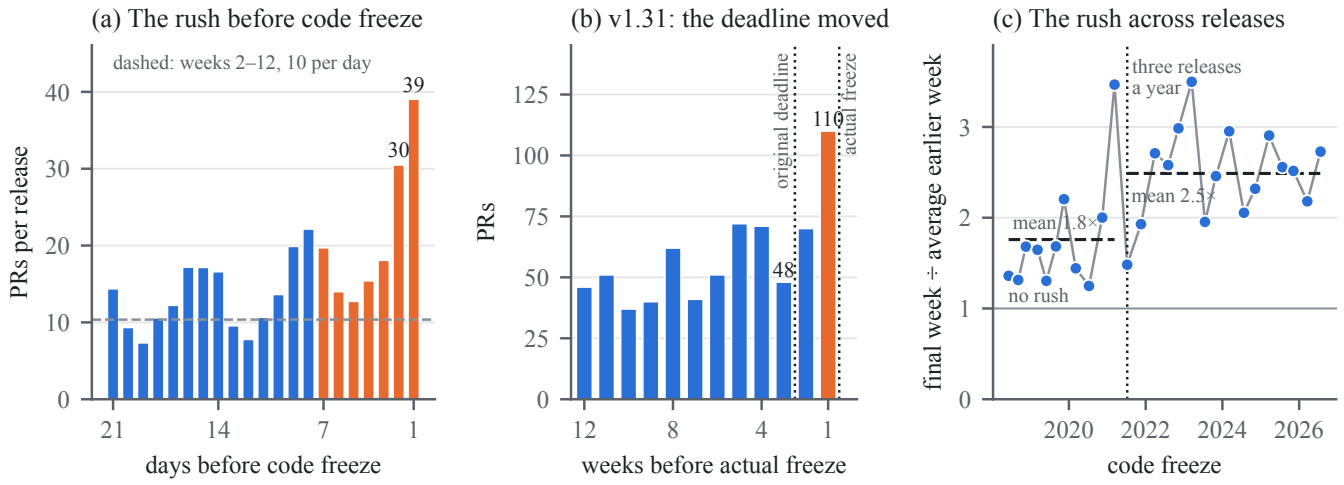


Fig. 2. The code-freeze rush in Kubernetes. (a) Pull requests per release by the day they became ready to merge, over the final three weeks before code freeze (27 releases, v1.11–v1.37); orange marks the final week. (b) v1.31 pull requests by week: the freeze was moved two weeks later, from the original deadline to the actual freeze. (c) Pull requests in each release's final week, divided by an average earlier week; dashed lines are the means before and after the move to three releases a year.

fewer defects than developers expect from it [36]. Our results suggest that the project's review process absorbs much of the rush, and that the remaining risk lies in what review cannot catch: problems that only show up after code has run for a while.

Counting defects after release: Many studies count defects in a window after release [15], and others relate the timing of a change to its defects [5], [6]. When the changes being compared reach the release at different times after they are made, a window that starts at the release mixes how often code fails with how soon its failures are noticed. In terms of Fig. 1, our findings suggest that merging late raises problems found after release through the soak path, not the code path. The shape of a time series can help tell these paths apart: deadline pressure predicts a jump at the deadline, while soak time predicts a gradual rise as the time to release shrinks. Studies that use post-release outcomes can also separate the two by following every change for the same window from its own start, and by reporting what was found before and after release.

VI. Threats to Validity

Construct validity: Our counts are lower bounds by design, since we count only problems that someone linked back to a pull request. One author hand-coded the fix links, so the coding reflects one person's judgment; to limit bias, the coding page hid whether a pull request was late. A revert shows that a change was pulled out, not always that it was faulty. Readiness dates when a pull request was approved, not when its work was done. We describe the kind and area of a change by its labels, which do not always match the code: in Kubernetes, labels match the changed code fully in 46.6% of pull requests [37]. We measure soak time as the days from merge to release, a period the project defines as one of testing and stabilization [3], [4]; we do not measure the testing itself.

Internal validity: Our estimates are causal only under the assumptions in Section III-C. The weighting and author comparisons adjust only for what we measured, so an unmeasured factor that makes a change both late and risky would bias them. We added the soak time analysis and the descriptive series of the rush after seeing our planned results, so we report them as exploratory. The timing prediction assumes that late and early problems take equally long to notice; the nearly identical discovery curves (Fig. 4(a)) support this assumption, but problems found after release may be noticed in other ways, for example by users.

VII. Future Work

Testing soak time directly: Our evidence for soak time comes from timing within each release. A direct test needs releases whose time from freeze to release differs; across 27 Kubernetes releases, such a test was too weak to tell either way. We are extending our analysis to other projects with time-based releases and code freezes, to test whether longer soak time moves problems back before the release.

Showing soak time to release teams: We plan to build tooling that shows the rush as it builds and flags merged changes that will reach a release with little soak time, so that release teams can target extra testing where it is most needed, and to evaluate it with release team members through interviews.

VIII. Data Availability

Our code, data, hand-coded links and results are available online [anonymous link].

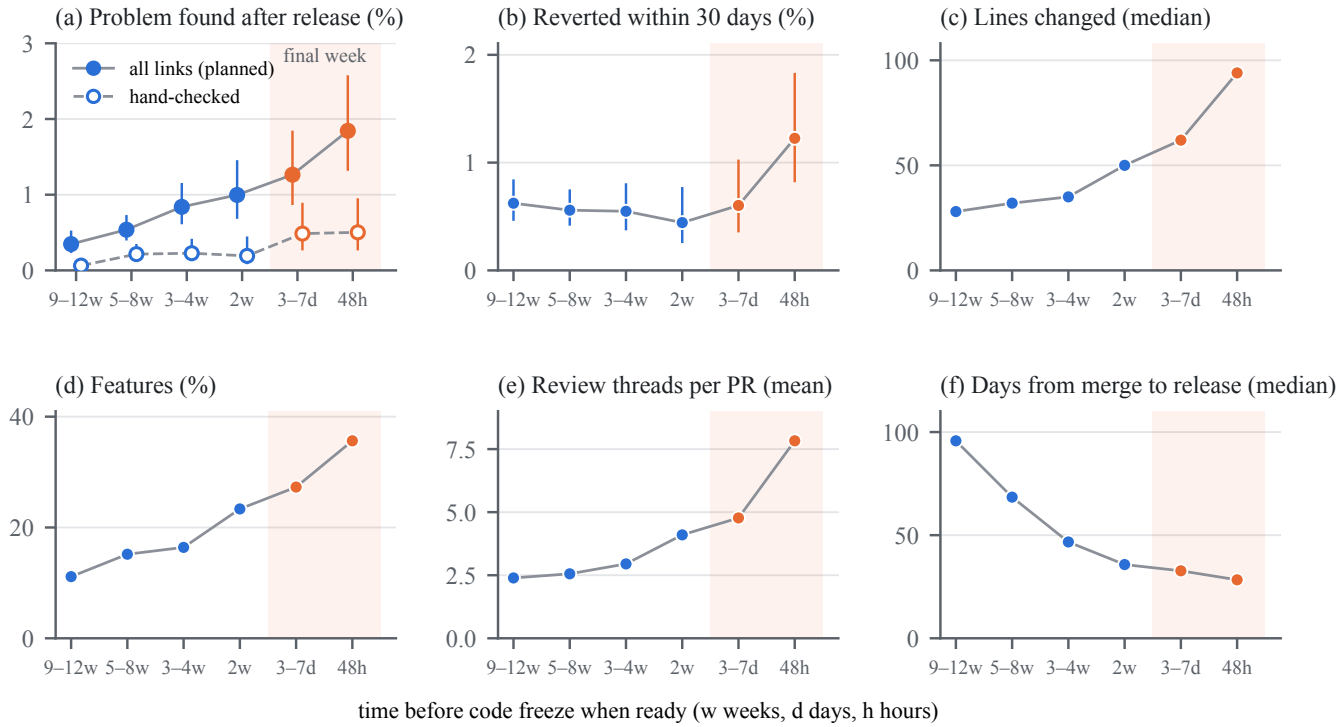


Fig. 3. How pull requests change as the freeze approaches, grouped by the time before code freeze at which they became ready (shaded: the final week). (a) Share with a problem found in the 90 days after release, for all links and for checked links. (b) Share reverted within 30 days of merge (checked links). (c–f) Lines changed, share of features, review threads, and days from merge to release. Bars are 95% confidence intervals.

References

[1] M. Michlmayr, B. Fitzgerald, and K.-J. Stol, "Why and how should open source projects adopt time-based releases?" *IEEE Software*, vol. 32, no. 2, pp. 55–63, 2015.

[2] Kubernetes SIG Release, "KEP-2572: Defining the Kubernetes release cadence," https://github.com/kubernetes/enhancements/tree/master/keps/sig-release/2572-release-cadence, 2021.

[3] ——, "Release phases," https://github.com/kubernetes/sig-release/blob/master/releases/release_phases.md, accessed October 2026.

[4] ——, "Kubernetes release cycle," https://kubernetes.io/releases/release/, accessed October 2026.

[5] J. Śliwerski, T. Zimmermann, and A. Zeller, "Don't program on fridays! how to locate fix-inducing changes," in *Softwaretechnik-Trends Band 25, Heft 2*, U. Kelter, Ed. Gesellschaft für Informatik e.V., 2005.

[6] J. Eyolfson, L. Tan, and P. Lam, "Do time of day and developer experience affect commit bugginess?" in *Proceedings of the 8th Working Conference on Mining Software Repositories*, ser. MSR '11. New York, NY, USA: Association for Computing Machinery, 2011, p. 153–162. [Online]. Available: https://doi.org/10.1145/1985441.1985464

[7] M. Kuutila, M. Mäntylä, U. Farooq, and M. Claes, "Time pressure in software engineering: A systematic review," *Information and Software Technology*, vol. 121, p. 106257, 2020. [Online]. Available: https://www.sciencedirect.com/science/article/pii/S0950584920300045

[8] F. Khomh, T. Dhaliwal, Y. Zou, and B. Adams, "Do faster releases improve software quality? an empirical case study of mozilla firefox," in *2012 9th IEEE Working Conference on Mining Software Repositories (MSR)*, 2012, pp. 179–188.

[9] B. Adams and S. McIntosh, "Modern release engineering in a nutshell – why researchers should care," in *2016 IEEE 23rd International Conference on Software Analysis, Evolution, and Reengineering (SANER)*, vol. 5, 2016, pp. 78–90.

[10] S. Aufiero, M. Vaccargiu, S. Bartolucci, F. Caccioli, and G. Destefanis, "Coordination at scale in large distributed development: The case of kubernetes," in *Proceedings of the 23rd International Conference on Mining Software Repositories*, ser. MSR '26. New York, NY, USA: Association for Computing Machinery, 2026, p. 98–109. [Online]. Available: https://doi.org/10.1145/3793302.3793342

[11] M. V. Mäntylä, B. Adams, F. Khomh, E. Engström, and K. Petersen, "On rapid releases and software testing: a case study and a semi-systematic literature review," *Empirical Softw. Engg.*, vol. 20, no. 5, p. 1384–1425, Oct. 2015. [Online]. Available: https://doi.org/10.1007/s10664-014-9338-4

[12] J. Śliwerski, T. Zimmermann, and A. Zeller, "When do changes induce fixes?" *SIGSOFT Softw. Eng. Notes*, vol. 30, no. 4, p. 1–5, May 2005. [Online]. Available: https://doi.org/10.1145/1082983.1083147

[13] D. A. da Costa, S. McIntosh, W. Shang, U. Kulesza, R. Coelho, and A. E. Hassan, "A framework for evaluating the results of the szz approach for identifying bug-introducing changes," *IEEE Transactions on Software Engineering*, vol. 43, no. 7, pp. 641–657, 2017.

[14] G. Rosa, L. Pascarella, S. Scalabrino, R. Tufano, G. Bavota, M. Lanza, and R. Oliveto, "Evaluating szz implementations through a developer-informed oracle," in *2021 IEEE/ACM 43rd International Conference on Software Engineering (ICSE)*, 2021, pp. 436–447.

[15] T. Zimmermann, R. Premraj, and A. Zeller, "Predicting defects for eclipse," in *Third International Workshop on Predictor Models in Software Engineering (PROMISE'07: ICSE Workshops 2007)*, 2007, pp. 9–9.

[16] S. Kim and E. J. Whitehead, "How long did it take to fix bugs?" in *Proceedings of the 2006 International Workshop on Mining Software Repositories*, ser. MSR '06. New York, NY, USA: Association for Computing Machinery, 2006, p. 173–174. [Online]. Available: https://doi.org/10.1145/1137983.1138027

[17] J. Shimagaki, Y. Kamei, S. McIntosh, D. Pursehouse, and N. Ubayashi, "Why are commits being reverted?: A comparative study of industrial and open source projects," in *2016 IEEE International Conference on Software Maintenance and Evolution (ICSME)*, 2016, pp. 301–311.

[18] S. McIntosh, Y. Kamei, B. Adams, and A. E. Hassan, "The impact of code review coverage and code review participation on software quality: a case study of the qt, vtk, and itk projects," in *Proceedings of the 11th Working Conference on Mining Software Repositories*, ser. MSR 2014.

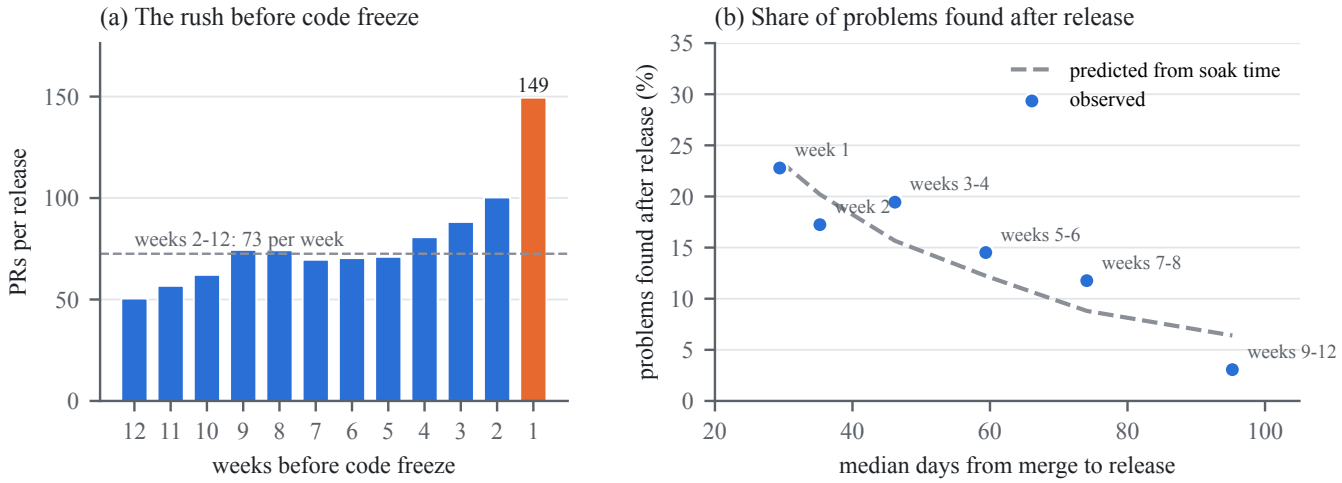


Fig. 4. Soak time. (a) Share of problems found within a given number of days of merge, for late (final week) and early (weeks 2–12) pull requests; dotted lines mark the median days from merge to release, and the dots the share already found by then. (b) Share of each week's problems found after release, against the median days from merge to release. The dashed line is the share predicted from soak time alone, using the other weeks' discovery times. Problems are checked links within 180 days of merge.

[[Elijah: panel (a) of fig_soak.pdf currently shows the rush bar chart (pull requests per week before the freeze), not the share of problems found by days since merge that this caption and Section IV-B describe. Please regenerate panel (a); panel (b) also has an overlapping "week 2" label.]]

New York, NY, USA: Association for Computing Machinery, 2014, p. 192–201. [Online]. Available: https://doi.org/10.1145/2597073.2597076

[19] J. Siebert, "Applications of statistical causal inference in software engineering," *Information and Software Technology*, vol. 159, p. 107198, 2023. [Online]. Available: https://www.sciencedirect.com/science/article/pii/S0950584923000526

[20] C. A. Furia, R. Torkar, and R. Feldt, "Towards causal analysis of empirical software engineering data: The impact of programming languages on coding competitions," *ACM Trans. Softw. Eng. Methodol.*, vol. 33, no. 1, Nov. 2023. [Online]. Available: https://doi.org/10.1145/3611667

[21] R. Pietrantuono, L. Giamattei, S. Russo, J. Siebert, and N. Walkinshaw, "Causal software engineering: A vision and roadmap," in *Proceedings of the 34th ACM International Conference on the Foundations of Software Engineering*, ser. FSE Companion '26. New York, NY, USA: Association for Computing Machinery, 2026, p. 1346–1350. [Online]. Available: https://doi.org/10.1145/3803437.3805585

[22] Kubernetes SIG Release, "Release schedules, v1.11–v1.37," https://github.com/kubernetes/sig-release/tree/master/releases, accessed October 2026.

[23] Y. Kamei, E. Shihab, B. Adams, A. E. Hassan, A. Mockus, A. Sinha, and N. Ubayashi, "A large-scale empirical study of just-in-time quality assurance," *IEEE Transactions on Software Engineering*, vol. 39, no. 6, pp. 757–773, 2013.

[24] J. Siebert, J. Frattini, H.-M. Heyn, and R. Pietrantuono, "Introduction to causal inference for software engineering," in *Proceedings of the 34th ACM International Conference on the Foundations of Software Engineering*, ser. FSE Companion '26. New York, NY, USA: Association for Computing Machinery, 2026, p. 52–53. [Online]. Available: https://doi.org/10.1145/3803437.3804898

[25] H. Bang and J. M. Robins, "Doubly robust estimation in missing data and causal inference models," *Biometrics*, vol. 61, no. 4, pp. 962–973, 12 2005. [Online]. Available: https://doi.org/10.1111/j.1541-0420.2005.00377.x

[26] P. M. Aronow and C. Samii, "Estimating average causal effects under general interference, with application to a social network experiment," *The Annals of Applied Statistics*, vol. 11, no. 4, pp. 1912 – 1947, 2017. [Online]. Available: https://doi.org/10.1214/16-AOAS1005

[27] Kubernetes v1.31 Release Team, "[ANNOUNCEMENT] v1.31 code freeze deadline moved to July 24th," Kubernetes dev mailing list, https://groups.google.com/a/kubernetes.io/g/dev/c/m_p2IsuDm-Q, 2024, posted 28 June 2024.

[28] Z. Zeng, Y. Zhang, H. Zhang, and L. Zhang, "Deep just-in-time defect prediction: how far are we?" in *Proceedings of the 30th ACM SIGSOFT International Symposium on Software Testing and Analysis*, ser. ISSTA 2021. New York, NY, USA: Association for Computing Machinery, 2021, p. 427–438. [Online]. Available: https://doi.org/10.1145/3460319.3464819

[29] P. Thongtanunam and A. E. Hassan, "Review dynamics and their impact on software quality," *IEEE Transactions on Software Engineering*, vol. 47, no. 12, pp. 2698–2712, 2021.

[30] S. Ananthanarayanan, M. S. Ardekani, D. Haenikel, B. Varadarajan, S. Soriano, D. Patel, and A.-R. Adl-Tabatabai, "Keeping master green at scale," in *Proceedings of the Fourteenth EuroSys Conference 2019*, ser. EuroSys '19. New York, NY, USA: Association for Computing Machinery, 2019. [Online]. Available: https://doi.org/10.1145/3302424.3303970

[31] L. Da Silva, P. Borba, and A. Pires, "Build conflicts in the wild," *Journal of Software: Evolution and Process*, vol. 34, no. 4, p. e2441, 2022. [Online]. Available: https://onlinelibrary.wiley.com/doi/abs/10.1002/smr.2441

[32] M. Yan, X. Xia, D. Lo, A. E. Hassan, and S. Li, "Characterizing and identifying reverted commits," *Empirical Softw. Engg.*, vol. 24, no. 4, p. 2171–2208, Aug. 2019. [Online]. Available: https://doi.org/10.1007/s10664-019-09688-8

[33] R. Pietrantuono, S. Russo, and K. S. Trivedi, "Software reliability and testing time allocation: An architecture-based approach," *IEEE Transactions on Software Engineering*, vol. 36, no. 3, pp. 323–337, 2010.

[34] D. A. D. Costa, S. Mcintosh, C. Treude, U. Kulesza, and A. E. Hassan, "The impact of rapid release cycles on the integration delay of fixed issues," *Empirical Softw. Engg.*, vol. 23, no. 2, p. 835–904, Apr. 2018. [Online]. Available: https://doi.org/10.1007/s10664-017-9548-7

[35] T. Savor, M. Douglas, M. Gentili, L. Williams, K. Beck, and M. Stumm, "Continuous deployment at facebook and oanda," in *Proceedings of the 38th International Conference on Software Engineering Companion*, ser. ICSE '16. New York, NY, USA: Association for Computing Machinery, 2016, p. 21–30. [Online]. Available: https://doi.org/10.1145/2889160.2889223

[36] A. Bacchelli and C. Bird, "Expectations, outcomes, and challenges of modern code review," in *2013 35th International Conference on Software Engineering (ICSE)*, 2013, pp. 712–721.

[37] M. Vaccargiu, S. Aufiero, S. Bartolucci, R. de Souza Santos, R. Tonelli, and G. Destefanis, "Efficiency for experts, visibility for newcomers: A case study of label-code alignment in Kubernetes," in *Proceedings of the 30th International Conference on Evaluation and Assessment in Software Engineering (EASE)*, 2026, preprint: arXiv:2603.24501.